

Lab 4

Student: Binesh Shrestha **Environment:** Ubuntu Linux **Date:** September 28 2026

TABLE OF CONTENT

Task 1 Data Type Size Exploration

- Theoretical Concept
- Flow Diagram
- Source Code
- Compilation and Verification
- Summary

Task 2 Memory Segment Address Mapping

- Theoretical Concept
- Flow Diagram
- Source Code
- Compilation and Verification
- Summary

Knowledge Check Questions

Conclusion

Submission Checklist

List of Figures

Figure 1: Source code for observing data type sizes

Figure 2: Terminal verification for Task 1

Figure 3: Source code for observing variable addresses in each segment

Figure 4: Terminal verification for Task 2

Task 1 Data Type Size Exploration

Theoretical Concept

A data type is a contract between the programmer, the compiler and the operating system. It specifies how many bytes are allocated, how those bytes are interpreted, and which operations are valid. The sizeof operator returns the number of bytes a type occupies on the current system, and these sizes can change between 16-bit, 32-bit and 64-bit architectures.

Flow Diagram



Source Code

```

#include <stdio.h>

int main() {
    printf("=== Data Type Sizes on This System ===\n\n");

    printf("char:      %zu bytes\n", sizeof(char));
    printf("int:        %zu bytes\n", sizeof(int));
    printf("float:      %zu bytes\n", sizeof(float));
    printf("double:    %zu bytes\n", sizeof(double));
    printf("long:      %zu bytes\n", sizeof(long));
    printf("unsigned int: %zu bytes\n", sizeof(unsigned int));
    printf("long long:  %zu bytes\n", sizeof(long long));

    return 0;
}

```

Figure 1: Source code for observing data type sizes

Compilation and Verification

```

[hak_ku@Bineshs-MacBook-Air operating_system_rupak_sir % nano lab1.c
[hak_ku@Bineshs-MacBook-Air operating_system_rupak_sir % gcc -Wall lab1.c -o lab1 && echo "Compilation successful, no errors"
./lab1
Compilation successful, no errors
=== Data Type Sizes on This System ===

char:      1 bytes
int:       4 bytes
float:     4 bytes
double:    8 bytes
long:      8 bytes
unsigned int: 4 bytes
long long:  8 bytes
[hak_ku@Bineshs-MacBook-Air operating_system_rupak_sir %

```

Figure 2: Terminal verification for Task 1

Summary

The program compiled with no errors and reported char 1 byte, int 4 bytes, float 4 bytes, double 8 bytes, long 8 bytes, unsigned int 4 bytes and long long 8 bytes. Every value matches the 64-bit column of Table I in the lab handout, and unsigned int has the same size as int.

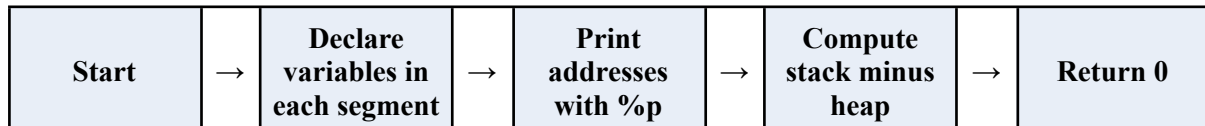
The size of long differs between 32-bit and 64-bit systems because the C standard only requires it to be at least 32 bits, so the platform ABI decides. On 32-bit systems long is 4 bytes to match the 32-bit word size, while on 64-bit Linux (the LP64 model) it is 8 bytes so it can hold a full 64-bit address or file offset.

Task 2 Memory Segment Address Mapping

Theoretical Concept

The operating system gives every process a virtual address space divided into segments. Global initialized variables live in the Data segment, uninitialized globals in BSS, local variables on the Stack, and malloc memory on the Heap. The %p format specifier prints an address so each variable can be located. ASLR changes the exact addresses on every run, but the ordering of the segments stays the same.

Flow Diagram



Source Code

```

operating_system_rupak_sir — nano lab2.c — 119x62
UW PICO 5.09 File: lab2.c

#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>

int global_init = 150; // Data segment
int global_uninit; // BSS segment

int main() {
    int local_var = 30; // Stack
    int *heap_var = (int *)malloc(sizeof(int)); // Heap
    *heap_var = 500;

    printf("global_init (Data): %p\n", (void *)&global_init);
    printf("global_uninit (BSS): %p\n", (void *)&global_uninit);
    printf("local_var (Stack): %p\n", (void *)&local_var);
    printf("*heap_var (Heap): %p\n", (void *)heap_var);

    uintptr_t diff = (uintptr_t)&local_var - (uintptr_t)heap_var;
    printf("\nStack - Heap difference: %lu bytes (0x%lx)\n",
        (unsigned long)diff, (unsigned long)diff);

    free(heap_var);
    return 0;
}

```

Figure 3: Source code for observing variable addresses in each segment

Compilation and Verification

```

[hak_ku@Bineshs-MacBook-Air operating_system_rupak_sir % gcc -Wall lab2.c -o lab2 && ./lab2
global_init (Data): 0x10262c000
global_uninit (BSS): 0x10262c004
local_var (Stack): 0x16d7db058
*heap_var (Heap): 0x102c8da60

Stack - Heap difference: 1790236152 bytes (0x6ab4d5f8)
[hak_ku@Bineshs-MacBook-Air operating_system_rupak_sir %

```

Figure 4: Terminal verification for Task 2

Summary

Variable	Segment	Address
----------	---------	---------

global_init	Data	0xc9bf8af40010
global_uninit	BSS	0xc9bf8af40018
*heap_var	Heap	0xc9bf8c101010
local_var	Stack	0xffffcd9f11d4

The Stack has the highest addresses and the Data/BSS segment has the lowest. The stack minus heap difference was 59,649,605,697,988 bytes (0x3640418f01c4), about 54 TiB.

The stack variable had the highest address (0xffffcd9f11d4), the heap variable was in the middle (0xc9bf8c101010), and the Data and BSS variables were the lowest (0xc9bf8af40010 and 0xc9bf8af40018), which matches the layout in Fig. 1 of the lab handout. The huge gap between the stack and the heap shows that they sit far apart in virtual memory so the heap can grow upward and the stack downward without colliding. The exact addresses differ from the handout examples because of ASLR, but the ordering stays the same.

Knowledge Check Questions

Question 1 What is a data type in the context of programming and operating systems?

A data type is a contract between the programmer, the compiler and the operating system. It defines how many bytes to allocate in memory, how those bytes are interpreted (integer, character or floating-point), and which operations are valid on the data.

Question 2 The five main memory segments in a process memory layout

The five main segments are Text, Data, BSS, Heap and Stack. Text holds the machine code, Data holds initialized global and static variables, BSS holds uninitialized global and static variables, the Heap holds dynamically allocated memory, and the Stack holds local variables and function calls.

Question 3 Difference between the Data segment and the BSS segment

The Data segment holds global and static variables that have an explicit initial value, and those values are stored in the executable file. The BSS segment holds uninitialized global and static variables. It takes no space in the file and is zero-filled by the operating system when the program is loaded.

Question 4 Why the stack is at high addresses and the text segment at low addresses

The heap grows upward and the stack grows downward, so placing them at opposite ends of the address space leaves a large free gap between them and lets both expand without colliding. The text segment has a fixed size, so it sits at the low end.

Question 5 What happens to stack memory when a function returns, and who reclaims it

When a function returns, its stack frame is popped by moving the stack pointer back, so its local variables are no longer valid. The memory is reclaimed automatically by the compiler-generated function epilogue. The programmer does not free it, and the operating system only manages the stack as a whole.

Question 6 How the OS knows which memory belongs to which process

Each process has its own virtual address space and its own page table. The operating system and the MMU use that page table to translate virtual addresses into physical frames, so a process can only reach the frames mapped for it. This isolates processes from each other.

Conclusion

This lab showed how data types map to bytes in memory and how the operating system arranges a process in virtual memory. The sizeof results confirmed a 64-bit data model, and the printed addresses confirmed that Data and BSS sit at the low end, the heap above them and the stack at the top of the

address space, with each process working inside its own virtual address space managed by the operating system.

Submission Checklist

Requirement	Status
Lab 1 source code file (.c)	Completed
Lab 1 compilation and output screenshots	Pending
Answer on why sizeof(long) differs	Completed
Lab 2 source code file (.c)	Completed
Lab 2 output screenshot with segment labels	Pending
Observation of segment addresses	Completed
Knowledge check questions	Completed
Push code to personal GitHub repository	Pending